Video 05 — Program execution: narration script and teaching outline

Video file: BlueImpulse_Video05_Program_Execution.mp4
Viewing position: 5 of 5
Duration: 10:00 (600 seconds)
Format: MP4, 1920 × 1080, 60 fps
File size: 45,852,242 bytes (45.85 MB)

Prerequisites: Basic arithmetic and the idea that memory locations have addresses. No assembly-language experience is assumed.

Learning goals:

- Trace fetch, decode, and execute
- Give each register its job
- Follow a four-instruction program

Chapter and subtopic connections:

- Chapter 1 — The Fundamentals / Fetch / decode / execute
- Chapter 1 — The Fundamentals / From source code to execution
- Chapter 1 — The Fundamentals / Trace a tiny program

Apply in the learning system: Complete ACT-01 in the Study Guide / Markdown activity file. Use Test Bank questions TF01–TF04, MC01–MC04 and OE01 for review. The six-chapter PPT deck introduces the same chapter concepts through classroom discussion.

Optional interactive companion: ROVERLAB lets you apply these ideas by building a robot, connecting its network and comparing stimulus-response rules. Play at https://roverlab-blue-impulse.vercel.app and use the ROVERLAB activity in the Study Guide. It is a simplified rule-and-feedback model, not a neural network or the numerical Q-learning algorithm taught in Video 01.

Materials: Paper and a calculator are sufficient. The optional supporting Python demonstration uses Python 3.9 or newer with no third-party packages or account.

```
python3 supporting/BlueImpulse_Demonstrations.py --topic execution
```

Chapter timestamps and teaching route

Time	Teaching step	Chapter / subtopic
00:00–00:32	Where does seven go?	Chapter 1 — The Fundamentals / Fetch / decode / execute
00:32–01:11	Today's route	Chapter 1 — The Fundamentals / Fetch / decode / execute
01:11–02:00	Before the processor executes	Chapter 1 — The Fundamentals / From source code to execution
02:00–02:50	Use the recipe analogy	Chapter 1 — The Fundamentals / Fetch / decode / execute
02:50–03:54	Fetch one instruction	Chapter 1 — The Fundamentals / Fetch / decode / execute
03:54–04:41	Read the four instructions	Chapter 1 — The Fundamentals / Trace a tiny program
04:41–05:51	Single-step the worked program	Chapter 1 — The Fundamentals / Trace a tiny program
05:51–06:44	Value or address?	Chapter 1 — The Fundamentals / Trace a tiny program
06:44–07:31	The next address can change	Chapter 1 — The Fundamentals / Fetch / decode / execute
07:31–08:25	Change one instruction	Chapter 1 — The Fundamentals / Trace a tiny program
08:25–09:32	Instructions change machine state	Chapter 1 — The Fundamentals / Fetch / decode / execute
09:32–10:00	Pip completes the shift	Chapter 1 — The Fundamentals / Fetch / decode / execute

Using this script

The following narration is the exact measured cue text for the revised recording. Cue timestamps refer to the video timeline; small gaps include intentional thinking time and transition space. The voice is synthetic (Kokoro ONNX, am_michael), not a recording of a student. The full teaching outline states the purpose, visual focus and learner action alongside the words spoken.

00:00–00:32 — Where does seven go?

Content reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Teaching / visual direction: Keep Pip's problem visible while the opening question establishes why the concept matters. Key visual labels: 3 / + 4 / STORE 7.

00:00.150–00:03.755: Three deliveries, plus four. Where does seven go?

00:03.755–00:09.215: Pip's tiny demonstration program must add two values and save the result.

00:09.215–00:19.394: The processor follows instruction meanings and changes stored state. It does not understand the whole program like a reader understands a story.

00:19.394–00:23.980: Have you ever wondered how a line of code becomes a visible result?

00:23.980–00:31.169: Chapter one, The Fundamentals, lets us trace the registers and follow every important state change.

00:32–01:11 — Today's route

Content reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Teaching / visual direction: Reveal the learning goals in the order spoken, then retain a small progress cue. Key visual labels: FETCH / DECODE / EXECUTE.

00:33.020–00:40.080: You will explain fetch, decode, and execute, and give each teaching-model register a clear job.

00:40.080–00:46.053: You will trace four instructions and distinguish an immediate value from a memory address.

00:46.053–00:55.741: You will also connect source code, translators, and machine instructions without oversimplifying how modern language implementations work.

00:55.741–01:00.626: Welcome to the tutorial on program execution by Blue Impulse.

01:00.626–01:09.652: Have a small table ready with columns for the program counter, current instruction, accumulator, and memory location two hundred.

01:11–02:00 — Before the processor executes

Content reference: Chapter 1 — The Fundamentals / From source code to execution

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: SOURCE / TRANSLATOR / MACHINE CODE.

01:11.628–01:20.399: The subtopic From source code to execution explains that machine code contains the instructions a processor executes.

01:20.399–01:30.215: An assembler translates assembly mnemonics and labels. A compiler produces another representation, often native code or bytecode.

01:30.215–01:36.081: An interpreter runs operations through a runtime, sometimes working with bytecode.

01:36.081–01:44.297: The same language can have compiled and interpreted implementations, and modern systems may combine several techniques.

01:44.297–01:50.824: Avoid the blanket claim that every interpreter translates exactly one source line at a time.

01:50.824–01:58.312: Our demonstration uses invented assembly notation so that every state change remains easy to see.

02:00–02:50 — Use the recipe analogy

Content reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: PC / MAR / MDR / CIR / ACC.

02:00.672–02:07.775: Use a recipe to remember the register jobs. The program counter marks the next recipe line.

02:07.775–02:15.838: The memory address register selects the location. The memory data register carries the item read from memory.

02:15.838–02:24.822: The current instruction register holds the instruction being followed. The accumulator is a temporary mixing bowl for arithmetic.

02:24.822–02:31.584: We abbreviate those names as P C, M A R, M D R, C I R, and A C C.

02:31.584–02:39.712: The analogy helps you remember the jobs, but these are electronic storage locations inside a processor model.

02:39.712–02:48.141: Keep the jobs distinct: an address points to a location, while an instruction or value is the content being handled.

02:50–03:54 — Fetch one instruction

Content reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: ADDRESS / INSTRUCTION / DECODE / EXECUTE.

02:50.759–02:57.777: Follow the fetch sequence. First copy the address in the program counter to the memory address register.

02:57.777–03:03.216: Read the instruction at that memory location into the memory data register.

03:03.216–03:13.267: Advance the program counter so it identifies the next instruction address, then copy the fetched instruction into the current instruction register.

03:13.267–03:21.245: The control unit can now decode the operation and its operand, and the processor executes the required work.

03:21.245–03:29.052: In this simplified model, every instruction occupies one word, so the program counter advances by one.

03:29.052–03:39.593: Real instruction sizes and exact timing depend on the architecture. Our table is a teaching model, not every processor's wiring diagram.

03:39.593–03:51.180: An address identifies a place. A value is the information stored or processed there. Keeping these roles separate makes the register trace much easier to follow.

03:54–04:41 — Read the four instructions

Content reference: Chapter 1 — The Fundamentals / Trace a tiny program

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: LOAD #3 / ADD #4 / STORE 200 / HALT.

03:54.411–04:02.218: In the subtopic Trace a tiny program, the first instruction sits at address one hundred: LOAD, hash three.

04:02.218–04:11.309: The hash sign denotes an immediate value in this invented notation, so that instruction puts the number three into the accumulator.

04:11.309–04:19.244: Address one hundred and one contains ADD, hash four. Address one hundred and two contains STORE, two hundred.

04:19.244–04:24.513: Address one hundred and three contains HALT, which ends this teaching program.

04:24.513–04:32.811: The number two hundred in STORE identifies a memory location. It is not the value to put into the accumulator.

04:32.811–04:39.274: Before running anything, predict what that memory location will contain when the program finishes.

04:41–05:51 — Single-step the worked program

Content reference: Chapter 1 — The Fundamentals / Trace a tiny program

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: PC / CIR / ACC / MEMORY[200].

04:41.889–04:50.596: Let's single-step. Initially, the program counter is one hundred. Fetch the LOAD instruction and advance to one hundred and one.

04:50.596–04:56.206: Decode and execute LOAD, hash three. The accumulator now contains three.

04:56.206–05:04.269: Next fetch ADD, hash four from address one hundred and one. The program counter advances to one hundred and two.

05:04.269–05:12.378: Execution adds four to the accumulator's three, giving seven. Memory location two hundred has not been written yet.

05:12.378–05:22.557: Fetch STORE, two hundred. The program counter advances to one hundred and three, and execution copies seven into memory location two hundred.

05:22.557–05:30.727: The accumulator still contains seven. STORE copies the value; it does not turn the accumulator into an address.

05:30.727–05:39.711: Finally fetch HALT. In this simplified trace, the fetch advances the counter to one hundred and four before execution stops.

05:39.711–05:47.756: The runnable Python model prints these same states and register transfers, so you can reproduce the trace yourself.

05:51–06:44 — Value or address?

Content reference: Chapter 1 — The Fundamentals / Trace a tiny program

Teaching / visual direction: Keep the supplied question legible during the thinking interval; reveal the answer only after the pause. Key visual labels: VALUE: 7 / ADDRESS: 200 / COPY.

Learner question: After STORE 200: ACC = ? and memory[200] = ?

Thinking interval: 06:04.850–06:09.850. The viewer may pause for longer.

05:51.280–05:59.304: Pause and check your prediction. After the STORE instruction, what are the accumulator and memory location two hundred?

05:59.304–06:04.850: Also ask yourself whether STORE, two hundred means storing the number two hundred.

06:09.850–06:18.429: The accumulator remains seven, and memory location two hundred receives seven. Two hundred is the destination address.

06:18.429–06:26.322: The hash sign in our LOAD and ADD operands marks literal values. The STORE operand has a different role.

06:26.322–06:33.830: This distinction prevents a common tracing mistake: confusing where data lives with the value stored there.

06:33.830–06:41.620: Use this model's instruction meanings before applying assumptions from a different programming language or processor.

Expected learning-check result: ACC = 7; memory[200] = 7

06:44–07:31 — The next address can change

Content reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: NEXT ADDRESS / BRANCH / REQUIRED EFFECT.

06:44.498–06:50.214: A branch instruction may replace the normal next value of the program counter.

06:50.214–06:57.915: That is how the instruction sequence can follow a decision or repeat a loop instead of always moving forward.

06:57.915–07:04.890: Our four-instruction program contains no branch, so the normal next addresses stay easy to follow.

07:04.890–07:14.472: Chapter three, Hardware, adds pipelining: real processors may overlap instructions while preserving the required program behaviour.

07:14.472–07:21.106: Overlapping the implementation does not make the logical effects of a program arbitrary.

07:21.106–07:29.556: For a basic trace, first understand the defined instruction effects, then study how a specific processor implements them.

07:31–08:25 — Change one instruction

Content reference: Chapter 1 — The Fundamentals / Trace a tiny program

Teaching / visual direction: Keep the supplied question legible during the thinking interval; reveal the answer only after the pause. Key visual labels: LOAD #3 / ADD #9 / STORE 200.

Learner question: LOAD #3; ADD #9; STORE 200

Thinking interval: 07:46.161–07:52.161. The viewer may pause for longer.

07:31.976–07:39.485: Your turn. Change ADD, hash four to ADD, hash nine, while keeping LOAD, hash three and STORE, two hundred.

07:39.485–07:46.161: What value should reach memory location two hundred? Work through the accumulator before we reveal it.

07:52.161–07:59.734: LOAD gives three. Adding nine gives twelve. STORE copies twelve into memory location two hundred.

07:59.734–08:05.835: That is a small example of testing a prediction by changing one input to the program.

08:05.835–08:15.331: The included simulator accepts a different add value, prints the trace, and checks that the final memory result matches the arithmetic.

08:15.331–08:22.733: Try another value and explain each register's role aloud as the instruction moves through the cycle.

Expected learning-check result: memory[200] = 3 + 9 = 12

08:25–09:32 — Instructions change machine state

Content reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: 3 + 4 / ACC = 7 / MEMORY[200] = 7.

08:25.715–08:34.465: Recap the lesson. Source code reaches execution through a translator or runtime appropriate to its implementation.

08:34.465–08:44.899: The program counter identifies the next instruction. Fetch brings it from memory, decode interprets it, and execute performs its operation.

08:44.899–08:55.163: The memory address register selects a location, the memory data register carries data, and the current instruction register holds the operation.

08:55.163–09:02.437: Our teaching accumulator went from three to seven, and STORE copied seven to memory location two hundred.

09:02.437–09:09.434: Addresses and values have different roles. Branches can change the normal next instruction address.

09:09.434–09:16.879: Complete the trace without watching the highlight. Understanding means predicting each state change yourself.

09:16.879–09:28.895: Pip's seven is now a stored result because STORE copied the accumulator value to the chosen memory address. The instruction determined both what changed and what stayed the same.

09:32–10:00 — Pip completes the shift

Content reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Teaching / visual direction: Resolve the opening problem and use a concise Blue Impulse end title, returning to the opening composition. Key visual labels: BLUE IMPULSE / FOLLOW THE SIGNAL.

09:32.498–09:36.422: Pip's small problem now has an explanation you can test.

09:36.422–09:42.395: Use Chapter 1, The Fundamentals, and the named subtopics to revisit the reasoning.

09:42.395–09:51.848: The tutorial guide includes the full teaching steps and the checked practice answers. Run the supporting demonstration and change one input.

09:51.848–09:57.352: Blue Impulse. Follow the signal, predict the result, and check your explanation.

Supporting demonstration and limits

This invented one-word ISA is a teaching model. The # sign marks a literal value; STORE's operand is a destination address. Begin at PC 100, ACC 0 and memory[200] 0.

Fetched address	Instruction	PC after fetch	ACC after execution	memory[200]
100	LOAD #3	101	3	0
101	ADD #4	102	7	0
102	STORE 200	103	7	7
103	HALT	104	7	7

STORE copies the accumulator's 7 to location 200; it does not store 200 or change ACC to 200. Real instruction sizes, register names and fetch timing depend on the architecture.

Practice: Replace ADD #4 with ADD #9. Predict memory[200] and explain the intermediate accumulator value.

Checked answer: LOAD makes ACC 3; ADD makes ACC 12; STORE copies 12 to memory[200]. The final PC remains 104 in this model. Changing one input and predicting every state is a stronger check than copying a highlighted answer.